

OCI Generative AI

De la configuración al primer chat

Tutorial práctico paso a paso

Proyecto Sentinel AcademIA
lFunknown

20 de junio de 2026

¿Qué aprenderás en este tutorial?

Este documento es el **log de aprendizaje** completo de cómo configurar y usar OCI Generative AI Inference desde tu terminal Linux. Está pensado para que lo leas y lo practiques tú mismo, replicando los pasos en tu propia cuenta.

Lo que cubrimos:

- Por qué el estándar moderno es **chat** (no **text**)
- Cómo **Cada modelo** (Cohere vs Gemini) tiene un formato distinto
- La diferencia entre **OCID del modelo** y **alias** (y por qué importa)
- Errores reales que encontramos y cómo resolverlos
- SDK Python **y** CLI: diferencias, cuándo usar cada uno
- Ejercicios prácticos al final

Disclaimer: Oracle + IA cambian constantemente

OCI Generative AI es un servicio **en evolución rápida**. Lo que era verdad en enero puede no serlo en junio:

- Modelos nuevos aparecen cada mes (Llama 4, Grok 4, Gemini 2.5...)
- Alias como `cohere.command-latest` pueden **redirigir** a otro modelo
- APIs deprecadas (`generate-text` → `chat`)
- Regiones se expanden, OCIDs cambian de formato

Lección: para producción, prefiere **OCID** sobre **alias**. Para aprender, los alias están bien.

1. Introducción: el cambio de paradigma `generate-text` → `chat`

OCI Generative AI ofrece dos grandes categorías de operaciones:

Operación	Sustituye	Cuándo usar
<code>generate-text</code>	(legacy)	Modelos viejos, deprecated
<code>chat</code>	ESTÁNDAR	Todo modelo nuevo

1.1. ¿Por qué chat y no text?

- **Historial:** chat mantiene contexto conversacional (lista de mensajes con roles)
- **Multimodalidad:** chat acepta imágenes, archivos, audio (en modelos que lo soportan)
- **Tool calling:** chat permite que el modelo invoque funciones externas
- **RAG nativo:** chat integra búsqueda en bases de conocimiento Oracle
- **El SDK deprecation:** cualquier intento de `generate-text` con modelos nuevos retorna:

Error típico:

Model cohere.command-latest does not support TextGeneration, only Chat.

1.2. El cambio de mentalidad: de “prompt” a “messages”

El formato text espera un solo string:

```
1 prompt = "Resume este artículo en una oración"
2 response = client.generate_text(
3     inference_request=inference_request_with_prompt=prompt
4 )
```

Listing 1: generate-text (obsoleto)

El formato chat espera una lista de mensajes con roles:

```
1 messages = [
2     {"role": "SYSTEM", "content": [{"type": "TEXT", "text": "Eres un
3     asistente conciso."}]},
4     {"role": "USER", "content": [{"type": "TEXT", "text": "Resume
5     este artículo"}]},
6 ]
7 response = client.chat(chat_request=CohereChatRequest(message="..."))
```

Listing 2: chat (estándar actual)

Conclusión: en este tutorial **siempre** usaremos chat.

2. Requisitos previos

2.1. Herramientas a instalar

Herramienta	Versión mínima	Para qué
oci (CLI)	3.0+	Llamadas desde terminal
Python	3.10+	SDK de OCI
oci SDK (pip)	2.130+	Llamadas programáticas

2.1.1. Instalación de OCI CLI (vía instalador oficial de Oracle)

El instalador de Oracle empaqueta Python 3.14 y la CLI juntas en un solo lugar (`~/lib/oracle-cli/bin/`). Esto evita contaminar tu Python del sistema.

```
1 # Descargar de
2   https://www.oracle.com/database/technologies/instant-client/linux-x86-64-downloads
3 # O usar pip del sistema:
```

```

3 pip3 install oci-cli
4
5 # Verificar
6 oci --version

```

Listing 3: Instalar OCI CLI en Linux

2.1.2. Instalación del SDK de OCI para Python

```

1 # Si usas el Python del sistema:
2 pip3 install oci
3
4 # Si usas el Python del instalador de Oracle (recomendado para aislar):
5 ~/lib/oracle-cli/bin/pip3 install oci

```

Listing 4: SDK de Python

2.2. Configurar el archivo ~/.oci/config

Este archivo contiene tus credenciales. **Nunca lo commitees a git.**

```

1 [DEFAULT]
2 user=ocid1.user.oc1..aaaaaa...      # OCID del usuario IAM
3 fingerprint=33:c3:5e:8d:...         # Huella de la API key
4 tenancy=ocid1.tenancy.oc1..aaaaaa... # OCID del tenancy (root)
5 region=us-chicago-1                # Región (debe coincidir con el
   modelo)
6 key_file=/home/usuario/.oci/private_key.pem

```

Listing 5: ~/.oci/config (formato INI)

Notas importantes:

- **region** debe ser la misma donde desplegaste el modelo (Chicago, Frankfurt, etc.)
- **key_file** debe tener permisos 600: `chmod 600 ~/.oci/private_key.pem` (el flag `~` se expande al HOME del usuario)
- **No** incluyas `compartment-id` en el config (lo pasamos por CLI/SDK)
- Si tienes varios perfiles, usa `[DEFAULT]`, `[ACADEMY]`, etc.

2.2.1. Obtener el OCID del compartment

```

1 oci iam compartment list --all \
2   --query 'data[?name=='SentinelAcademia'].{name:name, id:id}' \
3   --output table

```

Listing 6: Listar compartments

El OCID que devuelve (empieza con `ocid1.compartment.oc1..`) es el que pasamos a las llamadas LLM.

3. Concepto clave 1: cada modelo tiene un “request type” distinto

Este es el primer punto clave del tutorial y la razón por la que te confundiste al principio.

Regla de oro #1

El “serving mode” define qué tipo de objeto `chatRequest` debes usar.

- Modelo Cohere → `CohereChatRequest`
- Modelo Gemini → `GenericChatRequest` con contenido en formato array
- Mezclarlos → error 400 del API

3.1. Tabla resumen: SDK Python

Modelo (alias)	Request class	Campo de mensaje	Formato
<code>cohere.command-latest</code>	<code>CohereChatRequest</code>	<code>message</code> (string)	“Di hola”
<code>cohere.command-r-plus</code>	<code>CohereChatRequest</code>	<code>message</code> (string)	“Di hola”
<code>google.gemini-2.5-flash</code>	<code>GenericChatRequest</code>	<code>messages</code> (lista)	lista con roles

3.2. Tabla resumen: OCI CLI

Modelo	Sub-comando	Flag del mensaje
Cohere (cualquiera)	<code>chat-cohere-chat-request</code>	<code>--chat-request-message</code> (string)
Gemini (cualquiera)	<code>chat-generic-chat-request</code>	<code>--chat-request-messages</code> (JSON array)

Esto **NO** es documentado claramente en la doc oficial de OCI. Lo descubrimos probando.

4. Concepto clave 2: OCID del modelo vs alias (lo que más nos confundió)

Este es el **segundo punto clave**, igual de importante que el primero.

4.1. Las dos formas de identificar un modelo en OCI

	Alias (modelId)	OCID del modelo
Formato	<code>cohere.command-latest</code>	<code>ocid1.generativeaimodel.oc1.us-chicago-1.</code>
Estabilidad	Puede cambiar (Oracle lo apunta a otro modelo)	Inmutable, identifica una versión exacta
Para qué	Aprender, pruebas rápidas, documentación	Producción, deploys reproducibles
Dónde verlo	Cualquier doc o CLI	Solo en la Consola de OCI

4.2. ¿Por qué importa?

Ejemplo real: hoy `cohere.command-latest` apunta a Command R (ago-2024). Mañana Oracle podría actualizarlo a Command R2 (2026) sin avisarte. Tu código se comporta diferente sin que hayas tocado nada.

Con el OCID del modelo, eso **no pasa**: estás pinneado a esa versión exacta para siempre.

El campo `modelId` dentro de `--serving-mode` acepta **los dos** formatos:

```

1 # Con alias
2 --serving-mode
  '{"modelId":"cohere.command-latest","servingType":"ON_DEMAND"}'
3
4 # Con OCID
```

```

5 --serving-mode
  '{"modelId':"ocid1.generativeaimodel.oc1.us-chicago-1.amaaaaaa...',"servingType"

```

Listing 7: Misma llamada, dos formas de identificar el modelo

El servidor de OCI resuelve ambos al mismo modelo físico.

5. Cómo obtener el OCID de un modelo pre-built

Importante

La CLI de OCI no lista los modelos pre-built directamente. Los OCIDs solo se pueden obtener desde la **Consola Web de OCI**.

5.1. Paso a paso en la Consola

1. Abre <https://cloud.oracle.com/>
2. Menú → **Analytics & AI** → **Generative AI** → **Models**
3. Verás una tabla con todos los modelos disponibles en tu región
4. Cada fila tiene una columna **OCID** que empieza con `ocid1.generativeaimodel.oc1..`
5. Copia el OCID del modelo que quieras (ej. `cohere.command-latest`)

5.2. Verificación: ¿el OCID es correcto?

Una vez que tienes el OCID, pruébalo:

```

1 # Reemplaza <OCID> por el que copiaste
2 oci generative-ai-inference chat-result \
3   chat-cohere-chat-request \
4   -c "$COMPARTMENT_OCID" \
5   --serving-mode '{"modelId":"<OCID>","servingType":"ON_DEMAND"}' \
6   --chat-request-message "Di hola" \
7   --chat-request-max-tokens 20

```

Listing 8: Verificar que el OCID responde

Si devuelve texto → OCID válido. Si devuelve 404 → revisa que sea de la misma región.

6. Concepto clave 3: estrategia de estabilidad para producción

Regla de oro #2

Para **desarrollo y aprendizaje**: usa **alias** (más corto, más legible).
 Para **producción**: usa **OCID** (estable, reproducible).

6.1. Pattern recomendado en código

```

1 import os
2 import oci
3 from oci.generative_ai_inference import models as m
4

```

```

5 # OCID pinneado (tomado de la Consola de OCI una sola vez)
6 COHERE_OCID = os.environ["COHERE_MODEL_OCID"]
7 COMPARTMENT_OCID = os.environ["COMPARTMENT_OCID"]
8
9 def chat(prompt: str) -> str:
10     config = oci.config.from_file("~/oci/config", "DEFAULT")
11     client = oci.generative_ai_inference.GenerativeAiInferenceClient(
12         config,
13         service_endpoint="https://inference.generativeai.us-chicago-1.oci.oraclecloud.com"
14     )
15     details = m.ChatDetails(
16         compartment_id=COMPARTMENT_OCID,
17         serving_mode=m.OnDemandServingMode(model_id=COHERE_OCID), #
18         # OCID, no alias
19         chat_request=m.CohereChatRequest(message=prompt,
20             max_tokens=500, temperature=0.3),
21     )
22     return client.chat(details).data.chat_response.text

```

Listing 9: src/services/llm_client.py versión con OCID

6.2. Alias vs OCID: cuándo rotar

- **Cambias OCID:** cuando *tú* decides actualizar a un modelo nuevo (testeo, A/B)
- **Cambias alias:** cuando Oracle anuncia una deprecation del alias
- **Nunca cambias en runtime:** el OCID debe ser constante durante toda la vida del deploy

7. Guía 1: Cohere Command con Python SDK

7.1. Código funcional (verificado) dos versiones

7.1.1. Versión A: con alias (para aprender)

```

1 import oci
2 from oci.generative_ai_inference import models as m
3
4 config = oci.config.from_file("~/oci/config", "DEFAULT")
5 client = oci.generative_ai_inference.GenerativeAiInferenceClient(
6     config,
7     service_endpoint="https://inference.generativeai.us-chicago-1.oci.oraclecloud.com"
8 )
9
10 chat_request = m.CohereChatRequest(
11     message="Explica qu\'e es Python en una sola frase",
12     max_tokens=100,
13     temperature=0.3,
14     is_stream=False,
15 )
16
17 chat_details = m.ChatDetails(
18     compartment_id=config["tenancy"],
19     serving_mode=m.OnDemandServingMode(model_id="cohere.command-latest"),
20     # alias
21     chat_request=chat_request,
22 )

```

```

22 response = client.chat(chat_details)
23 print(response.data.chat_response.text)

```

Listing 10: test_cohere_chat.py con alias

7.1.2. Versión B: con OCID (para producción)

```

1  import oci
2  import os
3  from oci.generative_ai_inference import models as m
4
5  # OCID copiado UNA VEZ desde la Consola de OCI
6  COHERE_OCID = os.environ["COHERE_MODEL_OCID"]
7
8  config = oci.config.from_file("~/oci/config", "DEFAULT")
9  client = oci.generative_ai_inference.GenerativeAiInferenceClient(
10     config,
11     service_endpoint="https://inference.generativeai.us-chicago-1.oci.oraclecloud.com",
12 )
13 chat_request = m.CohereChatRequest(
14     message="Explica qu'è es Python en una sola frase",
15     max_tokens=100,
16     temperature=0.3,
17     is_stream=False,
18 )
19
20 chat_details = m.ChatDetails(
21     compartment_id=config["tenancy"],
22     serving_mode=m.OnDemandServingMode(model_id=COHERE_OCID), # OCID
23     chat_request=chat_request,
24 )
25
26 response = client.chat(chat_details)
27 print(response.data.chat_response.text)

```

Listing 11: test_cohere_chat.py con OCID

7.2. Anatomía de la respuesta

```

1 response.data # -> ChatResponse
2 # .chat_response -> CohereChatResponse
3 # .text -> "Python es un lenguaje..."
4 # .finish_reason -> "COMPLETE" | "MAX_TOKENS" | "ERROR"
5 # .usage -> {prompt_tokens, completion_tokens,
6   total_tokens}
7 # .model_id -> "cohere.command-latest"
8 # .model_version -> "1.7"

```

Listing 12: Inspeccionar la respuesta completa

7.3. Errores comunes con Cohere

Error 1: "Unrecognized keyword arguments: text"

Causa: Usaste `CohereChatRequest(text="...")` en vez de `message="..."`.

Solución: Cohere v1 usa `message` (singular, string), no `text`.

Error 2: "Model does not support TextGeneration, only Chat"

Causa: Intentaste usar `client.generate_text()` con un modelo Cohere nuevo.

Solución: Usa `client.chat()` siempre.

Error 3: "Compartment ID must be provided"

Causa: Pasaste `compartment_id` como string vacío o `None`.

Solución: Usa el OCID completo: `ocid1.compartment.oc1..aaaaaa...`

8. Guía 2: Cohere Command con CLI de OCI

8.1. Comando funcional (verificado) dos versiones

8.1.1. Versión A: con alias

```

1 COMPARTMENT_OCID="ocid1.compartment.oc1..aaaaaaaa4vvf5a7lznbrfxporgxgmk3mdanyktzsac
2
3 oci generative-ai-inference chat-result \
4   chat-cohere-chat-request \
5   -c "$COMPARTMENT_OCID" \
6   --serving-mode
7     '{"modelId":"cohere.command-latest","servingType":"ON_DEMAND"}' \
8   --chat-request-message "Di hola en una sola frase" \
9   --chat-request-max-tokens 30 \
10  --chat-request-temperature 0

```

Listing 13: Test Cohere CLI con alias

8.1.2. Versión B: con OCID (recomendado para producción)

```

1 # Variables de entorno (poner en ~/.bashrc)
2 export
3   COMPARTMENT_OCID="ocid1.compartment.oc1..aaaaaaaa4vvf5a7lznbrfxporgxgmk3mdanyktzsac
4 export
5   COHERE_MODEL_OCID="ocid1.generativeaimodel.oc1.us-chicago-1.aaaaaaaXXXXX..."
6   # <- de la Consola
7
8 oci generative-ai-inference chat-result \
9   chat-cohere-chat-request \
10  -c "$COMPARTMENT_OCID" \
11  --serving-mode
12    '{"modelId\":"$COHERE_MODEL_OCID\","servingType\":"ON_DEMAND\"}' \
13  --chat-request-message "Di hola en una sola frase" \
14  --chat-request-max-tokens 30 \
15  --chat-request-temperature 0

```

Listing 14: Test Cohere CLI con OCID

8.1.3. Atajo: alias de shell

Para no escribir todo el tiempo, crea este alias en tu `~/.bashrc`:


```

1 # Requiere: COMPARTMENT_OCID y COHERE_MODEL_OCID ya exportadas
2 alias ocichat='oci generative-ai-inference chat-result
3   chat-cohere-chat-request \
4   -c "$COMPARTMENT_OCID" \
5   --serving-mode
6     '{"modelId":"$COHERE_MODEL_OCID"},"servingType":"ON_DEMAND"}'
7
8 # Uso:
9 ocichat --chat-request-message "Di hola" --chat-request-max-tokens 30

```

Listing 15: ~/.bashrc

8.2. Output esperado

```

1 {
2   "data": {
3     "chat-response": {
4       "api-format": "COHERE",
5       "text": "¡Hola, cómo estás?",
6       "finish-reason": "COMPLETE",
7       "usage": {
8         "prompt-tokens": 7,
9         "completion-tokens": 8,
10        "total-tokens": 15
11      }
12    },
13    "model-id": "cohere.command-latest",
14    "model-version": "1.7"
15  }
16 }

```

Listing 16: Respuesta del CLI

8.3. Anatomía de los flags

Flag	Qué hace
chat-result	Sub-comando de alto nivel (imprime resultado, no job)
chat-cohere-chat-request	Especifica el formato del request (Cohere v1)
--compartment-id	OCID del compartment (no del tenancy)
--serving-mode	JSON con modelId y servingType
--chat-request-message	String con el prompt (Cohere es simple)
--chat-request-max-tokens	Límite de tokens en la respuesta

9. Guía 3: Gemini con CLI de OCI

9.1. Comando funcional (verificado)

```

1 COMPARTMENT_OCID="ocid1.compartment.oc1..aaaaaaa4vvf5a7lznbrfxporgxgmkr3mdanyktzsac
2
3 oci generative-ai-inference chat-result \
4   chat-generic-chat-request \
5   --compartment-id "$COMPARTMENT_OCID" \
6   --serving-mode
7     '{"modelId":"google.gemini-2.5-flash","servingType":"ON_DEMAND"}' \
8   --chat-request-messages
9     '[{"role":"USER","content":[{"type":"TEXT","text":"Di hola"}]}' \

```

```
8 --chat-request-max-tokens 20
```

Listing 17: Test Gemini desde terminal

9.2. ¿La trampa del formato!

Diferencia clave Cohere vs Gemini

Cohere CLI: `--chat-request-message` es un **string** simple.

Gemini CLI: `--chat-request-messages` es un **JSON array** con estructura anidada:

```
1 [
2   {
3     "role": "USER",
4     "content": [
5       {
6         "type": "TEXT",
7         "text": "Di hola"
8       }
9     ]
10  }
11 ]
```

Si pasas `content": "Di hola"` (string plano) → 400 “Please pass in correct format of request”.

9.3. ¿Por qué Gemini usa este formato raro?

- El campo `content` es una **lista** porque Gemini soporta **multimodalidad**
- Cada elemento de la lista es un **ContentPart** con `type`
- Los tipos posibles son: `TEXT`, `IMAGE`, `AUDIO`, etc.
- Hoy solo usamos `TEXT`, pero mañana puedes agregar imágenes

10. Comparación lado a lado: SDK vs CLI

10.1. Cohere Command

Python SDK	CLI
<code>m.CohereChatRequest(message=".. --chat-request-message "..."</code>	
<code>client.chat(chat_details)</code>	<code>oci ... chat-result</code>
<code>response.data.chat_response.text</code>	Filtrar JSON con <code>jq</code>

10.2. Gemini

Python SDK	CLI
<code>m.GenericChatRequest(messages=[</code>	<code>--chat-request-messages '[...]'</code>
<code>m.TextContent(text="...")</code>	<code>content": [{"type": "TEXT", "text": "..."}]</code>
<code>client.chat(chat_details)</code>	<code>oci ... chat-result</code>

11. Troubleshooting: errores que encontramos y cómo los resolvimos

11.1. Catálogo completo de errores

Error 1: "No such file or directory: private_key.pem"

Cuándo: oci config validate o primera llamada.

Causa: El key_file en el config (ruta ~/.oci/config) apunta a una ruta inexistente.

Solución:

```
1 ls -la ~/.oci/private_key.pem
2 # Si no existe, descargar de la consola de OCI:
3 # Profile -> API Keys -> Add API Key -> Download Private Key
4 chmod 600 ~/.oci/private_key.pem
```

Error 2: "Compartment ID is not valid"

Cuándo: Primera llamada con Python.

Causa: Pasaste el OCID entre corchetes [ocid1...] o con espacios.

Solución: OCID limpio, sin corchetes:

```
1 # MAL
2 COMP="[ocid1.compartment.oc1..aaa]"
3 # BIEN
4 COMP="ocid1.compartment.oc1..aaa..."
```

Error 3: "Entity with key cohere.command-r not found"

Cuándo: Listar modelos o primera llamada.

Causa: El nombre cohere.command-r sin sufijo de fecha no existe.

Solución: Usar el alias cohere.command-latest (siempre apunta al más nuevo) o la versión con fecha cohere.command-r-08-2024.

Error 4: "cohere.command-latest does not support TextGeneration"

Cuándo: Usar client.generate_text() con Cohere.

Causa: generate_text es para modelos viejos.

Solución: Usar client.chat() con CohereChatRequest siempre.

Error 5: "Unrecognized keyword arguments: text"

Cuándo: Crear CohereChatRequest.

Causa: Usar text="..." en vez de message="..."

Solución: Cohere v1 usa message (singular).

Error 6: "Please pass in correct format of request" (Gemini CLI)

Cuándo: Llamar Gemini con content como string.

Causa: Gemini requiere content como lista de ContentPart.

Solución:

```
1 # MAL
2 '{"role":"USER","content":"Di hola"}'
3
4 # BIEN
5 '{"role":"USER","content":[{"type":"TEXT","text":"Di hola"}]}'
```

Error 7: “404 Not Found” o timeout en llamada al API

Cuándo: Primera llamada desde una región nueva.

Causa: La región del `config` no coincide con la región donde está desplegado el modelo.

Solución: Verificar con:

```
1 oci iam region list --output table
```

y usar la misma región en `--region` o en el `config`.

12. Ejercicios prácticos

Haz estos ejercicios en orden, **anotando** los errores que te salen y cómo los resuelves.

12.1. Ejercicio 1: Tu primer chat con Cohere (5 min)

1. Abre tu terminal
2. Exporta la variable `COMPARTMENT_OCID` con tu OCID real
3. Copia y pega el comando de la sección 5.2
4. Verifica que el JSON de respuesta tiene `"text": "..."` con un saludo

Output esperado: Un JSON con `"finish-reason": "COMPLETE"` y un texto de respuesta.

12.2. Ejercicio 2: Cambia la temperatura (5 min)

1. Re-ejecuta el comando de Cohere con `--chat-request-temperature 1.5`
2. Compara la respuesta con la de temperatura 0
3. ¿Qué cambió? (Creatividad, longitud, formato)

12.3. Ejercicio 3: Prueba Gemini (5 min)

1. Cambia el sub-comando a `chat-generic-chat-request`
2. Cambia `--chat-request-message` por `--chat-request-messages` con el formato array
3. Ejecuta y compara la velocidad con Cohere

12.4. Ejercicio 4: Convierte el comando CLI a Python (15 min)

1. Toma el comando CLI de Cohere que ya te funciona
2. Tradúcelo a un script Python usando `oci.generative_ai_inference`
3. Guarda el script como `mi_primer_chat.py`
4. Ejecútalo y compara el output con el del CLI

Pista: Usa el código de la sección 4.1 (versión A o B) como base.

12.5. Ejercicio 5: Inserta el script en una función reutilizable (20 min)

1. Crea `src/services/llm_client.py`
2. Define una función `chat(prompt: str) → str` que oculte toda la complejidad
3. Escribe 3 tests con `pytest` que verifiquen:
 - Devuelve string no vacío
 - Falla limpiamente si el compartment OCID es inválido
 - Falla limpiamente si el modelo no existe

Este ejercicio es el que usaremos en Fase 1 del proyecto.

12.6. Ejercicio 6: Migra de alias a OCID (10 min)

1. Abre la Consola de OCI y obtén el OCID de `cohere.command-latest`
2. Exporta `COHERE_MODEL_OCID` en tu shell
3. Modifica tu función `chat()` para que use `COHERE_MODEL_OCID` en vez del alias
4. Re-ejecuta los tests del ejercicio 5
5. **Bonus:** crea un test que verifique que el modelo es estable (mismo OCID entre 2 llamadas)

Este patrón es el que usaremos en producción.

13. Resumen: lo que tienes que recordar

Checklist de memoria

1. **Estándar** = `chat`, nunca `generate_text`
2. **Cohere** → `CohereChatRequest` (string en `message`)
3. **Gemini** → `GenericChatRequest` (lista en `messages`)
4. **CLI Cohere** → `chat-cohere-chat-request` con string
5. **CLI Gemini** → `chat-generic-chat-request` con array JSON
6. **Región del config** = región del modelo
7. **Compartment OCID** va en cada llamada, no en el config
8. **Clave privada** con permisos 600, ubicada en `~/.oci/private_key.pem`
9. **Alias** = para aprender, **OCID** = para producción
10. **OCID del modelo solo en Consola**, no en CLI
11. **Oracle cambia:** revisa docs cada trimestre

14. Referencias

- <https://docs.oracle.com/en-us/iaas/Content/generative-ai/> — Documentación oficial de OCI Generative AI
- <https://docs.oracle.com/en-us/iaas/tools/python-sdk-examples/> — Ejemplos del SDK de Python
- <https://www.oracle.com/artificial-intelligence/generative-ai/large-language-models/> — Catálogo de modelos

Fin del tutorial. ¡Ahora a practicar!